

Phase 1 — Production Deployment

Runbook

You (operator) must execute these steps. I do not have network access to your production environment. Copy-paste these commands into your prod admin shell.

Estimated total time: ~**15 minutes** (most is the backup; deploy itself is ~2 minutes).

Step 0 — Pre-deploy snapshot (record for the audit trail)

```
DEPLOY_TS=$(date -u +%Y-%m-%dT%H:%M:%SZ)
GIT_COMMIT=$(cd /app && git rev-parse --short HEAD)
echo "Deploy timestamp (UTC): $DEPLOY_TS"
echo "Commit: $GIT_COMMIT"
```

Save these two values — you'll need them for the post-deploy report.

Step 1 — Verify `CRAVITOO_ENV=production` is set on the prod pod

```
# Inside the production backend pod:
echo "CRAVITOO_ENV=$CRAVITOO_ENV" # must print "production"
# Or grep the .env file the pod uses:
grep -E '^CRAVITOO_ENV' /app/backend/.env
```

If it is NOT `production`:

- Edit the production `.env` so the line reads exactly:

```
CRAVITOO_ENV="production"
```

Then restart the backend so it picks the new value.

Fail-secure reminder: if you accidentally leave the variable unset or invalid, the code falls back to `production` automatically. Setting it explicitly is still required by your approval conditions.

Step 2 — Full backup of the production MongoDB

```
BACKUP_DIR=/backups/phase1-pre-deploy-$(date -u +%Y%m%d-%H%M%S)
mkdir -p $BACKUP_DIR

# 1) Full DB dump
mongodump --uri="$PROD_MONGO_URL" --db="$PROD_DB_NAME" --out="$BACKUP_DIR" --

# 2) Verify the dump actually wrote
ls -lah $BACKUP_DIR/$PROD_DB_NAME/orders.bson.gz
ls -lah $BACKUP_DIR/$PROD_DB_NAME/users.bson.gz

# 3) Record the SHA256 so you can prove integrity at restore time
sha256sum $BACKUP_DIR/$PROD_DB_NAME/*.bson.gz > $BACKUP_DIR/sha256sums.txt
cat $BACKUP_DIR/sha256sums.txt

# 4) Quick row-count sanity check (no read of bodies)
mongosh "$PROD_MONGO_URL/$PROD_DB_NAME" --eval '
  ["orders","users","companies","sites","vendors"].forEach(c => print(c + ":
  '
```

Save `$BACKUP_DIR` and the sha256 output for the report.

Step 3 — Deploy the reviewed Phase 1 code

The Phase 1 changes are already in the preview environment under commit `$GIT_COMMIT`. Use **your existing deploy mechanism** (Emergent's "Deploy to production" button, your CI/CD pipeline, Docker image push + rolling restart — whichever you normally use).

The exact file list that should land on prod is:

File

Change type

backend/.env	+ CRAVITOO_ENV="production" (single new line — set per Step 1)
backend/env_config.py	NEW
backend/order_lifecycle.py	NEW
backend/models.py	modified
backend/routers/auth.py	modified
backend/routers/demo.py	modified
backend/server.py	modified
frontend/src/index.js	modified
frontend/src/context/ AuthContext.js	modified
frontend/src/pages/ LoginPage.js	modified
frontend/src/pages/ RegisterPage.js	modified
frontend/src/pages/master/ DemoControl.js	modified

Order of operations:

1. Deploy backend image first (it adds the env guard so the demo endpoints become 404 even if the old frontend still has the page).
2. Wait for backend health-check to go green.
3. Deploy frontend image.
4. **Do not** restart MongoDB.

Step 4 — Production smoke checklist (run this from your laptop)

```
cd /path/to/cravitoo/checkout
PROD_URL="https://app.cravitoo.com" \
ADMIN_EMAIL="admin@cravitoo.com" \
```

```
ADMIN_PASSWORD=<the_real_prod_admin_password>" \
python backend/scripts/phase1_prod_smoke.py
```

This script writes **zero** orders / payments / emails / users. It does generate **9 rows** in `audit_log` (one per privileged-role-escalation attempt — that's the security control we just installed *working as designed*). Every smoke row is tagged with an email pattern like `phase1_smoke_<YYYYMMDD_HHMMSS>+vendor@cravitoo.com` — you can purge them later with:

```
db.audit_log.deleteMany({ user_email: { $regex: "^phase1_smoke_" } });
```

...but there's no functional reason to.

Pass criteria: every line prints ✓ `PASS`. Final line reads 🟡 `Production smoke: ALL CHECKS PASS`.

If even one line shows ✗ `FAIL` → go to Step 6 (rollback).

Step 5 — Monitoring window (15 min after deploy)

In a second terminal, watch the prod backend logs for **15 minutes** after deploy:

```
# Adjust to your log-source (kubectl, journalctl, Datadog, etc.)
kubectl logs -f deployment/cravitoo-backend --tail=200 | \
  grep -Ei 'error|exception|traceback|429|500|503|cors|jwt|register_privileg
```

What you're looking for:

- **Expected (✓ benign):** `register_privileged_role_blocked` log lines from the smoke run.
- **Unexpected (✗ rollback signal):** any `500/503`, JWT decode tracebacks, unhandled exceptions on `/orders`, `/auth/login`, `/auth/refresh`, `/vendors`, `/me`, or repeated `429` storms that block legitimate users.

Also spot-check:

- Quick query on `audit_log` to confirm only your smoke entries appeared:

```
db.audit_log.find({
  action: "register_privileged_role_blocked",
  created_at: { $gte: new Date(Date.now() - 1000 * 60 * 60) }
```

```
}).count();
```

Should equal the number of `role-blocked` lines printed by the smoke script (9).

- Confirm `db.orders.find({ updated_at: { $gte: <deploy_ts> } }).count()` reflects **only natural traffic** during the smoke window, NOT artificial test writes.

Step 6 — Rollback procedure (do this immediately if Step 4 or 5 surfaces a regression)

```
# 1) Roll the deployment back to the previous commit/image
#     (Emergent platform: use the "Rollback" button to the snapshot you took
#     OR: re-deploy the prior image tag.

# 2) If the schema changed (it didn't in Phase 1), restore the DB from the d
mongorestore --uri="$PROD_MONGO_URL" --db="$PROD_DB_NAME" --drop --gzip $BAC

# 3) Smoke-test the rolled-back version with the SAME script
PROD_URL=... python backend/scripts/phase1_prod_smoke.py
#     Expected on a fully-rolled-back stack: ✗ FAILS on "role-blocked[*]" bec
#     pre-Phase-1 code accepts the privileged-role registration. That is the
#     that the rollback was complete.

# 4) Capture which step failed and ping me for triage.
```

Step 7 — Report fields to fill in

After Steps 1-5 complete, send me these values so I can compile the verification report:

Field	Where it comes from
Deployment UTC timestamp	Step 0 <code>\$DEPLOY_TS</code>
Deployed commit	Step 0 <code>\$GIT_COMMIT</code>
Backup directory + total size	Step 2 (<code>du -sh \$BACKUP_DIR</code>)
Backup sha256 of <code>orders.bson.gz</code>	<code>\$BACKUP_DIR/sha256sums.txt</code> line for orders
<code>CRAVITOO_ENV</code> value on prod	Step 1 grep output

Smoke checklist stdout	Step 4 — entire output (or just the summary line + any x FAILs)
Monitoring window — error count summary	Step 5 grep summary
Anomalous audit_log rows (if any)	Step 5 find() count
Any natural-traffic orders written during smoke window	Step 5 db.orders.count
Final verdict	PASS / ROLLED BACK

I'll then compile the verification report and store it next to the approval report.

Production data is read-only for me. The smoke script is read-only except for the 9 audit_log rows. Phase 2 is on hold until you confirm Phase 1 production status.